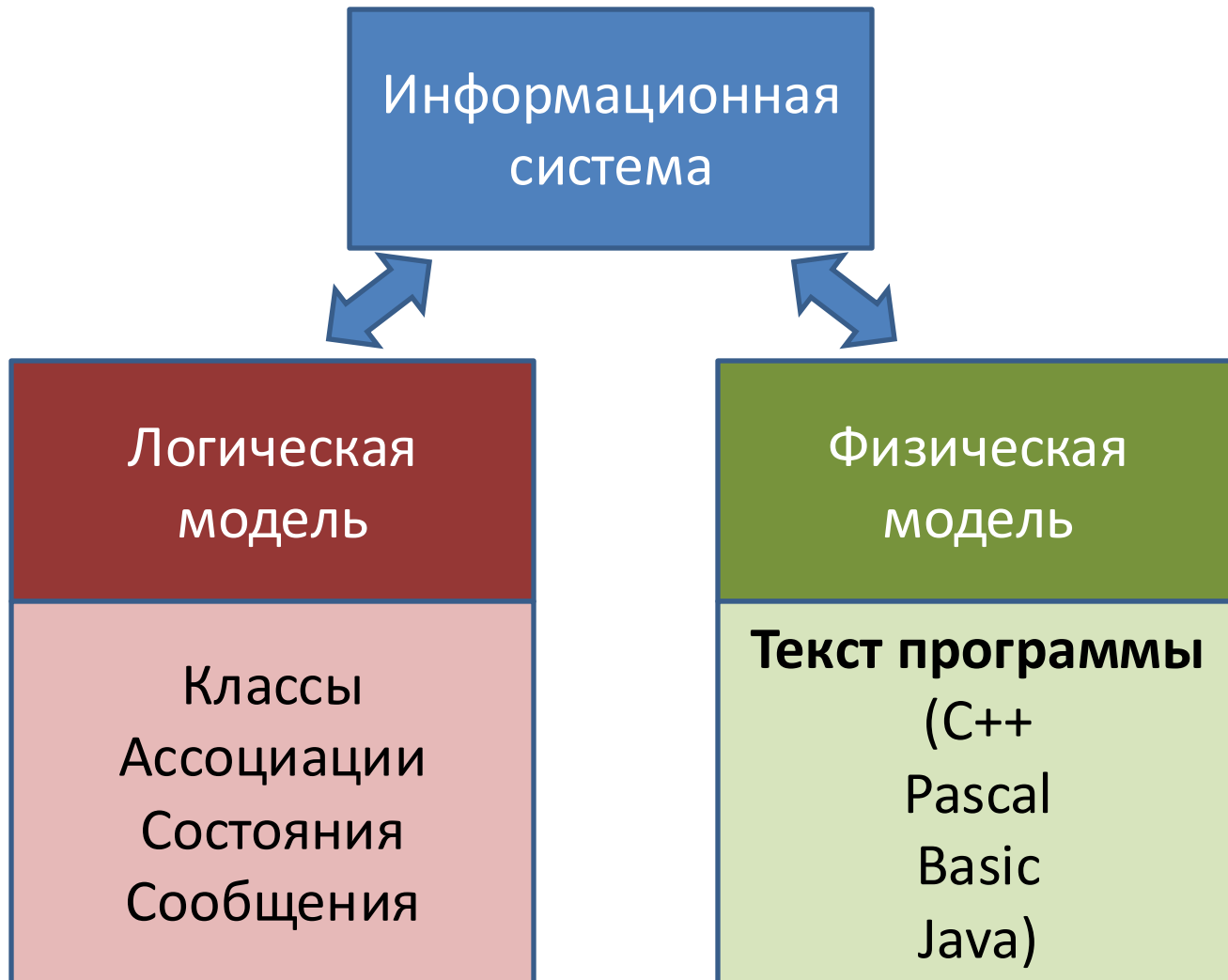


Проектирование информационных систем

Диаграммы компонентов
(component diagram)

Модели информационной системы



Классификация диаграмм UML

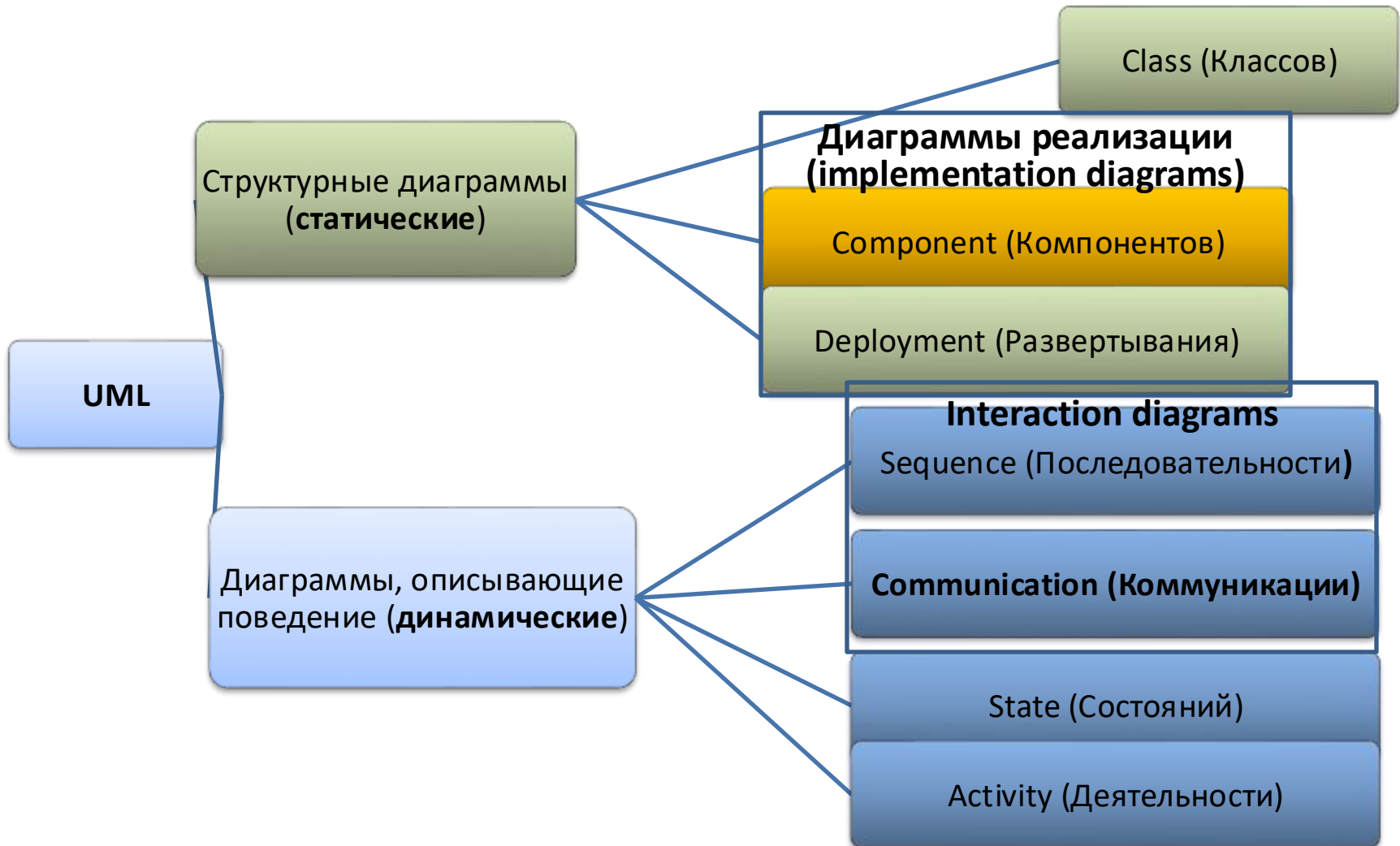
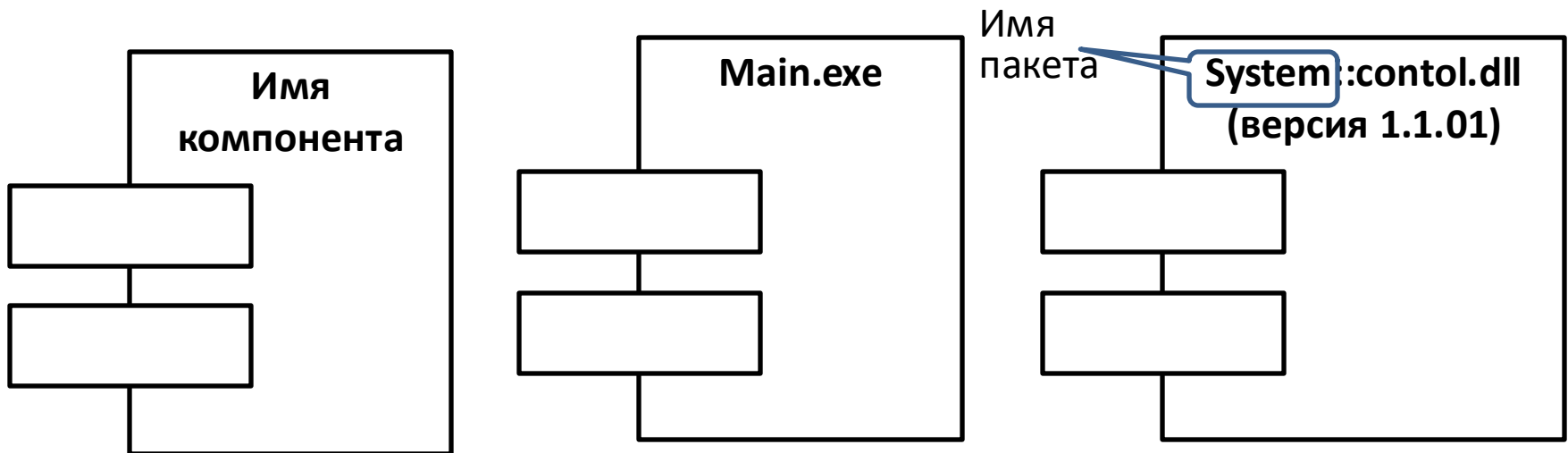


Диаграмма компонентов

- **Диаграмма компонентов** описывает особенности физического представления системы, определяет архитектуру разрабатываемой системы, установив зависимости между программными компонентами, в роли которых может выступать исходный и исполняемый код.
- Основными графическими элементами диаграммы компонентов являются компоненты, интерфейсы и зависимости между ними.

Компонент

- Для представления физических сущностей применяется специальный термин – **компонент (component)**.
- Для графического представления компонента может использоваться специальный символ – прямоугольник со вставленными слева двумя более мелкими прямоугольниками.
- Внутри объемлющего прямоугольника записывается имя компонента, и, возможно, некоторая дополнительная информация.
- Компонент может иметь собственные свойства: атрибуты и операции.



Имя компонента

- Отдельный компонент может быть представлен на уровне типа (класса) или на уровне экземпляра.
- Графическое изображение компонента в обоих случаях одинаковое, правила записи имени компонента несколько отличаются.
- Если компонент представляется на уровне типа, то в качестве его имени записывается только **имя типа с заглавной буквы**.
- Если же компонент представляется на уровне экземпляра, то в качестве его имени записывается
- **<имя компонента ':' имя типа>**
- При этом вся строка имени подчеркивается, хотя часто подчеркивание в литературе опускается.
- Имя экземпляра начинается со строчной буквы.
- Имена компонентов могут определяться особенностями языка программирования.
- В качестве простых имен принято использовать имена исполняемых файлов.

Виды компонентов

- Поскольку компонент представляет отдельный модуль кода, иногда его комментируют с указанием дополнительных графических символов, иллюстрирующих конкретные особенности его реализации.
- Эти дополнительные обозначения для примечаний не специфицированы в языке UML. Однако их применение упрощает понимание диаграммы компонентов, существенно повышая наглядность физического представления.
- В языке UML выделяют три вида компонентов.
 - компоненты развертывания, которые обеспечивают непосредственное выполнение системой своих функций: динамически библиотеки с расширением dll , Web-страницы с расширением html и файлы справки с расширением hlp .
 - компоненты-рабочие продукты: файлы с исходными текстами программ;
 - компоненты исполнения: файлы с расширением exe.

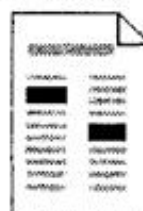
dialog.dll



ndex.html



context.hlp



main.cpp

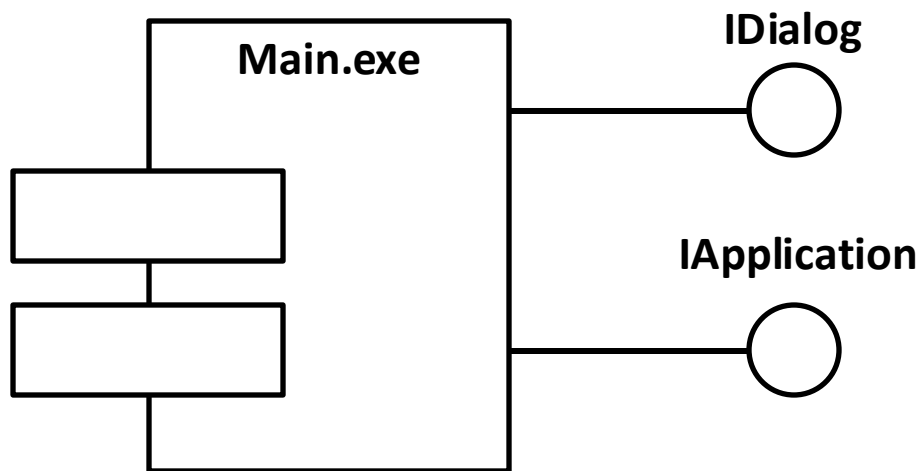


Виды компонентов

- Другой способ спецификации различных видов компонентов — явное указание стереотипа компонента перед его именем. В языке UML для компонентов определены следующие стереотипы:
- **Библиотека (library)** — представляется в форме динамической или статической библиотеки.
- **Таблица (table)** — представляется в форме таблицы базы данных.
- **Файл (file)** — представляется в виде файлов с исходными текстами программ.
- **Документ (document)** — представляется в форме документа.
- **Исполнимый (executable)** — компонент, который может исполняться в узле.

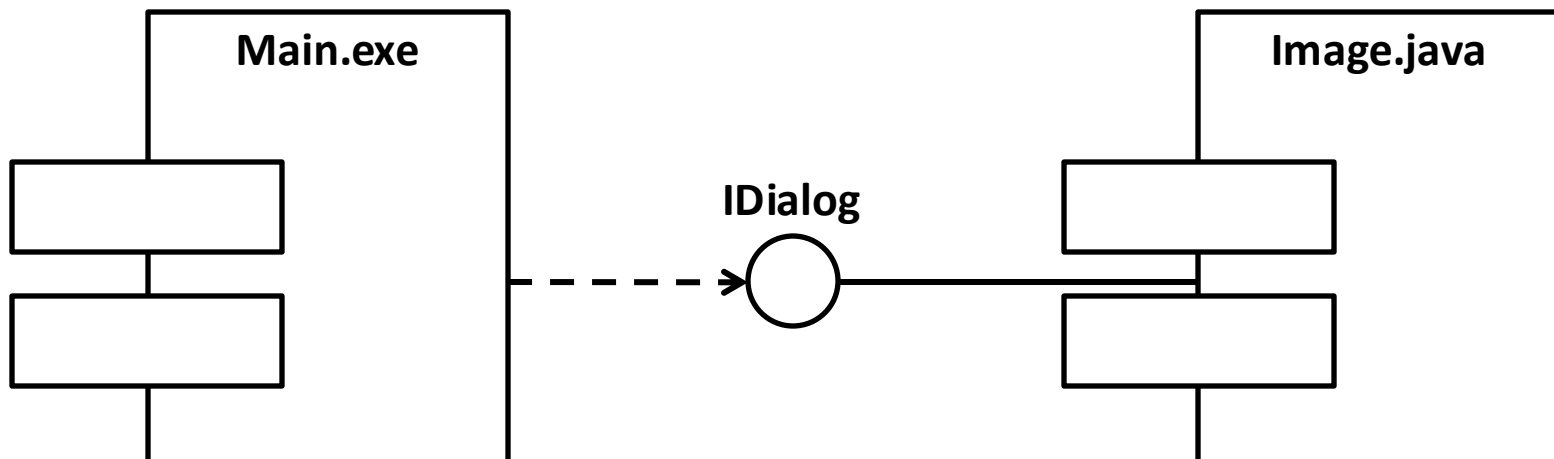
Интерфейсы

- Следующим элементом диаграммы компонентов являются интерфейсы.
- Интерфейс графически изображается окружностью, которая соединяется с компонентом отрезком линии без стрелок. При этом имя интерфейса, которое обязательно должно начинаться с заглавной буквы "I", записывается рядом с окружностью.
- Семантически линия означает реализацию интерфейса, а наличие интерфейсов у компонента означает, что данный компонент реализует соответствующий набор интерфейсов.



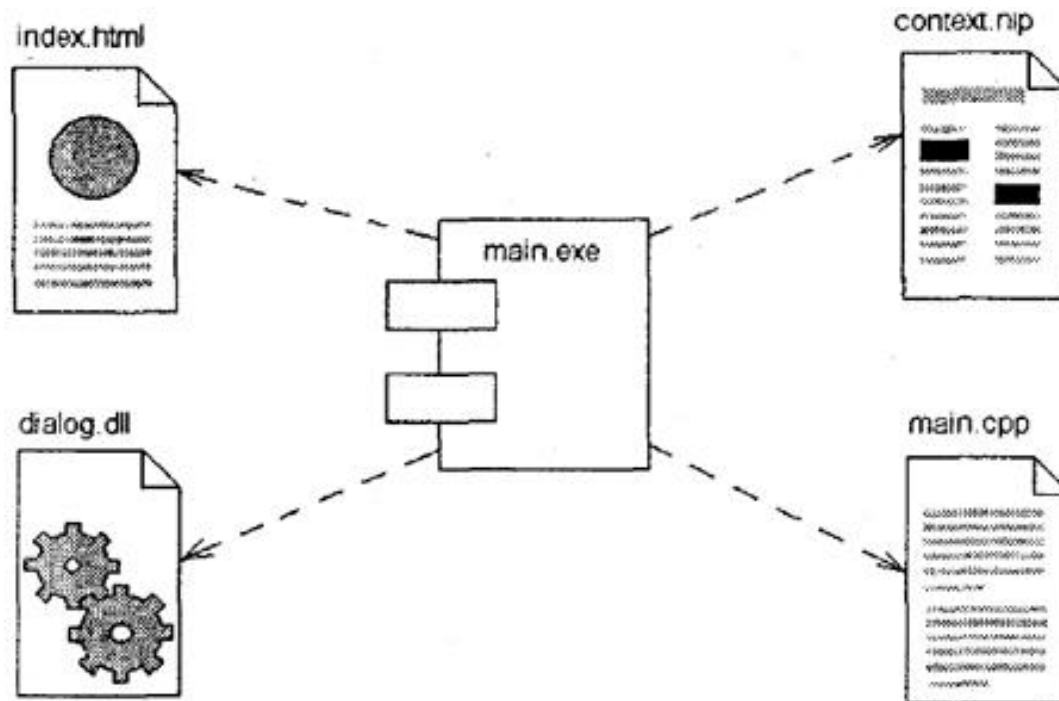
Зависимости

- Зависимость служит для представления факта наличия связи между элементами, когда изменение одного элемента модели оказывает влияние или приводит к изменению другого элемента модели.
- Отношение зависимости на диаграмме компонентов изображается пунктирной линией со стрелкой, направленной от клиента (зависимого элемента) к источнику (независимому элементу).
- Применительно к диаграмме компонентов зависимости могут связывать компоненты и импортируемые этим компонентом интерфейсы, а также различные виды компонентов между собой.
- Например, изображенный ниже фрагмент диаграммы компонентов представляет информацию о том, что компонент с именем "main.exe" зависит от импортируемого интерфейса I Dialog, который, в свою очередь, реализуется компонентом с именем "image.java". Для второго компонента этот же интерфейс является экспортируемым.



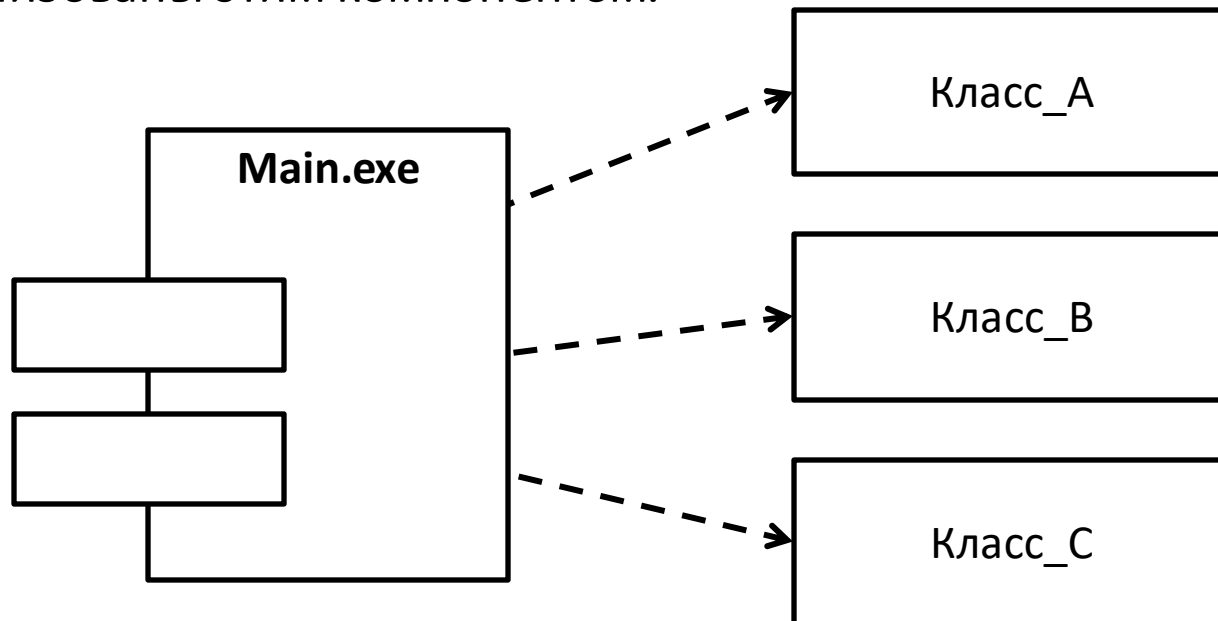
Зависимости

- Другим случаем отношения зависимости на диаграмме компонентов является отношение между различными видами компонентов .
- Наличие подобной зависимости означает, что внесение изменений в исходные тексты программ или динамические библиотеки приводит к изменениям самого компонента. При этом характер изменений может быть отмечен дополнительно.



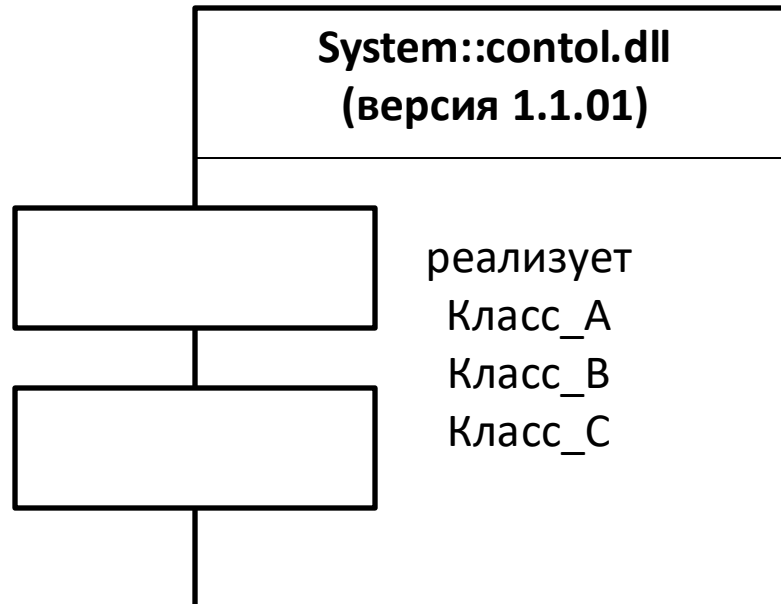
Зависимости

- На диаграмме компонентов могут быть представлены отношения зависимости между компонентами и реализованными в них классами.
- Эта информация имеет важное значение для обеспечения согласования логического и физического представлений модели системы.
- Изменения в структуре описаний классов могут привести к изменению компонента.
- В данном случае из диаграммы компонентов не следует, что классы реализованы этим компонентом.



Зависимости

- Если требуется подчеркнуть, что некоторый компонент реализует отдельные классы, то для обозначения компонента используется расширенный символ прямоугольника. При этом прямоугольник компонента делится на две секции горизонтальной линией.
- Верхняя секция служит для записи имени компонента, а нижняя секция — для указания дополнительной информации.



Зависимости

- Внутри символа компонента могут изображаться другие элементы графической нотации, такие как классы (компонент уровня типа) или объекты (компонент уровня экземпляра).
- В этом случае символ компонента изображается таким образом, чтобы вместить эти дополнительные символы.



Рекомендации

- Разработка диаграммы компонентов предполагает использование информации как о логическом представлении модели системы, так и об особенностях ее физической реализации.
- До начала разработки необходимо принять решения о выборе вычислительных платформ и операционных систем, на которых предполагается реализовывать систему, а также о выборе конкретных баз данных и языков программирования.
- После этого можно приступать к общей структуризации диаграммы компонентов. В первую очередь, необходимо решить, из каких физических частей (файлов) будет состоять программная система.

Рекомендации

- На этом этапе следует обратить внимание на такую реализацию системы, которая обеспечивала бы не только возможность повторного использования кода за счет рациональной декомпозиции компонентов, но и создание объектов только при их необходимости.
- Речь идет о том, что общая производительность программной системы существенно зависит от рационального использования ею вычислительных ресурсов.
- Для этой цели необходимо большую часть описаний классов, их операций и методов вынести в динамические библиотеки, оставив в исполняемых компонентах только самые необходимые для инициализации программы фрагменты программного кода.
- После общей структуризации физического представления системы необходимо дополнить модель интерфейсами и схемами базы данных.
- При разработке интерфейсов следует обращать внимание на согласование (стыковку) различных частей программной системы.
- Включение в модель схемы базы данных предполагает спецификацию отдельных таблиц и установление информационных связей между таблицами.

Рекомендации

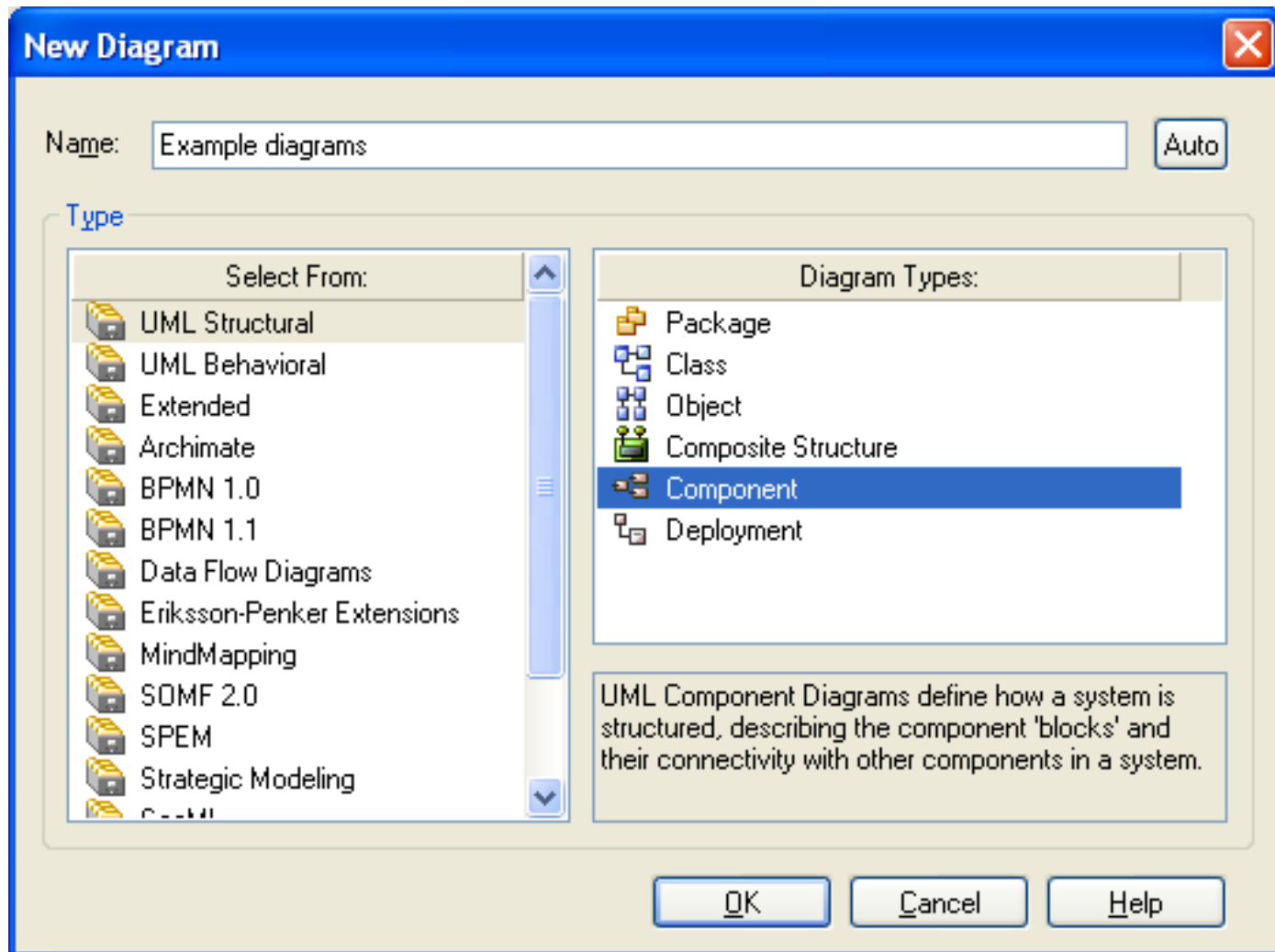
- Наконец, завершающий этап построения диаграммы компонентов связан с установлением и нанесением на диаграмму взаимосвязей между компонентами, а также отношений реализации.
- Эти отношения должны иллюстрировать все важнейшие аспекты физической реализации системы, начиная с особенностей компиляции исходных текстов программ и заканчивая исполнением отдельных частей программы на этапе ее выполнения. Для этой цели можно использовать различные виды графического изображения компонентов.
- При разработке диаграммы компонентов следует придерживаться общих принципов создания моделей на языке UML. В частности, в первую очередь необходимо использовать уже имеющиеся в языке UML компоненты и стереотипы. Для большинства типовых проектов этого набора элементов может оказаться достаточно для представления компонентов и зависимостей между ними.

Рекомендации

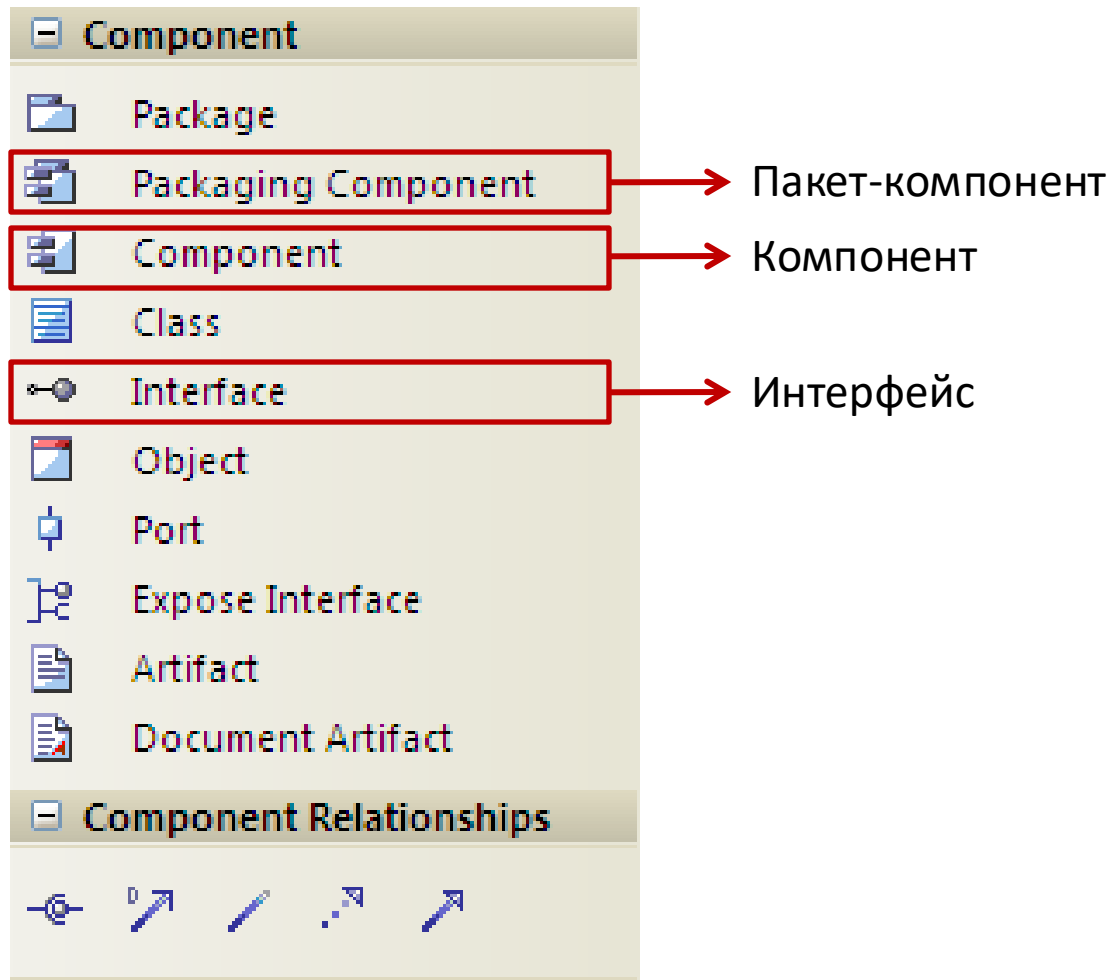
- Если же проект содержит некоторые физические элементы, описание которых отсутствует в языке UML, то следует воспользоваться механизмом расширения. В частности, использовать дополнительные стереотипы для отдельных нетиповых компонентов или помеченные значения для уточнения их отдельных характеристик.
- В заключение следует обратить внимание, что диаграмма компонентов, как правило, разрабатывается совместно с диаграммой развертывания, на которой представляется информация о физическом размещении компонентов программной системы по ее отдельным узлам.

Создание диаграммы компонентов

- New Diagram-> UML Structural-> Component

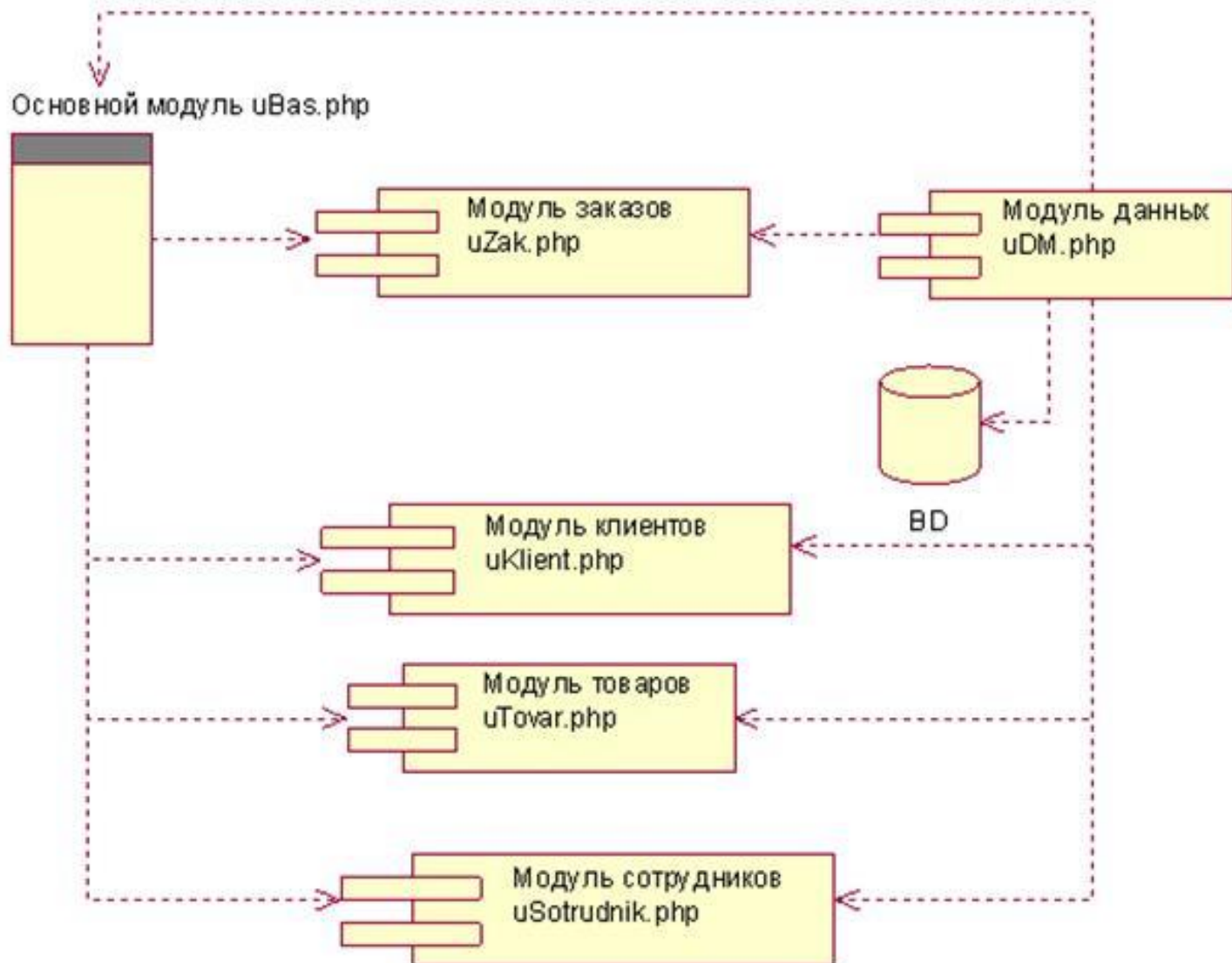


Создание диаграммы компонентов



- Для создания диаграммы используется инструмент «Component».

Пример



Порядок создания диаграмм

Создание

Диаграмма Классов

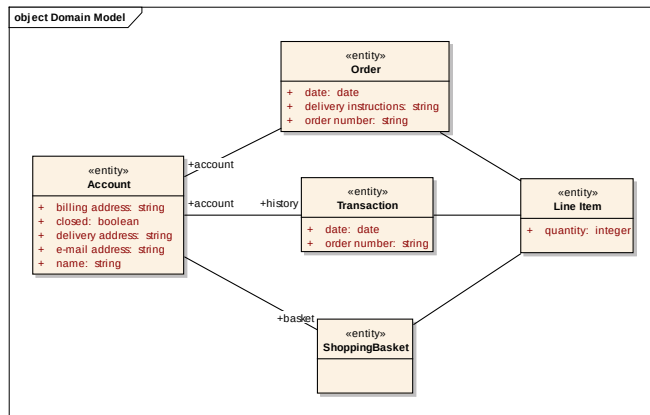
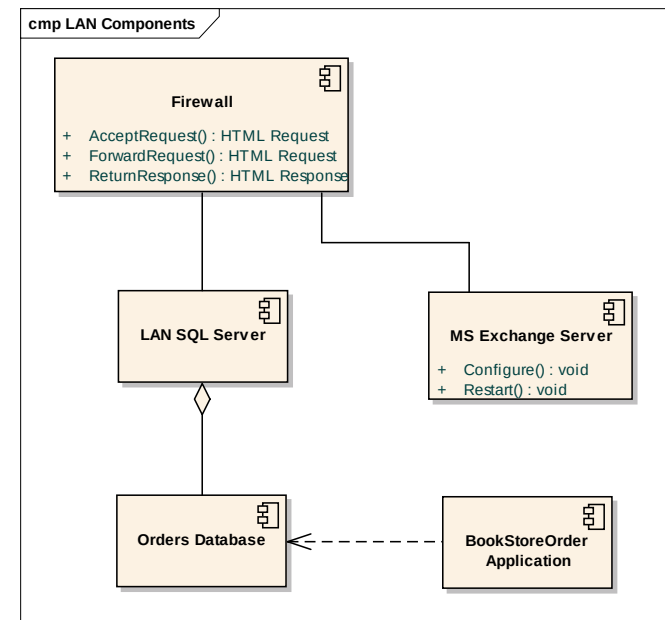
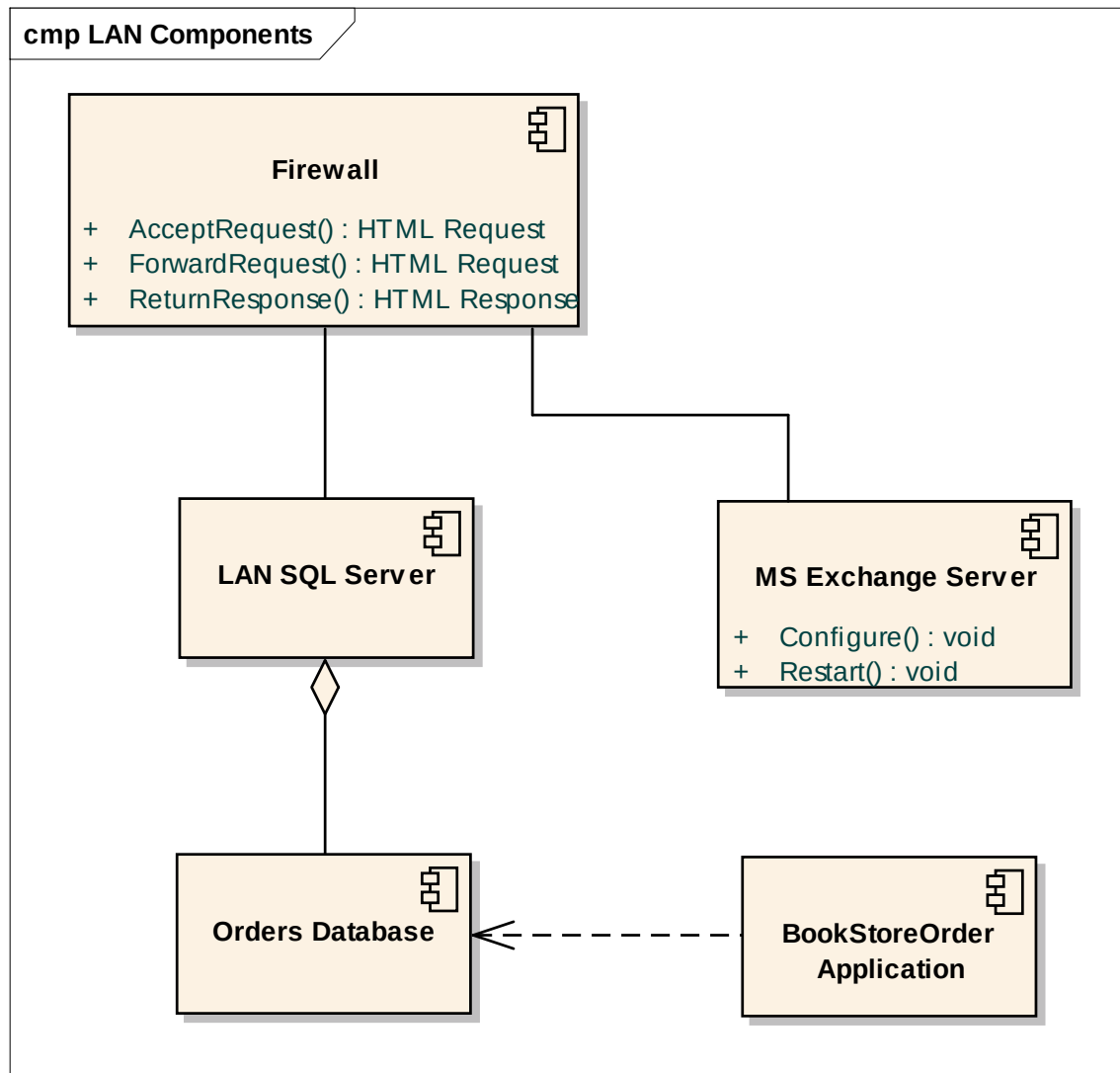


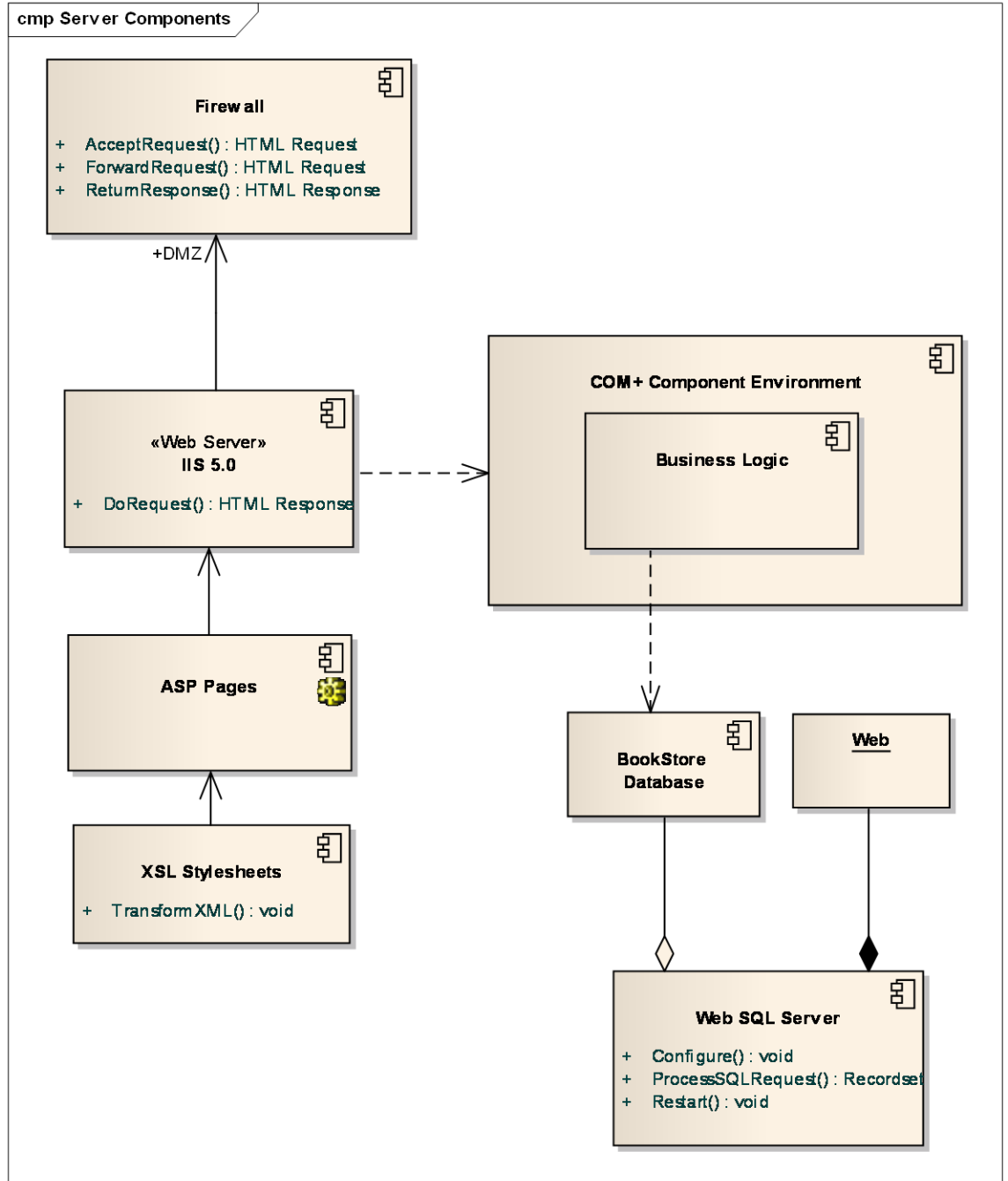
Диаграмма Компонентов



Пример



Пример



Рекомендации

